

SAE 3.01: BanalUML

Luka Salvo, Joey Rosenkranz, Robin Carette, Pacôme Terrier, Nathan Eyer

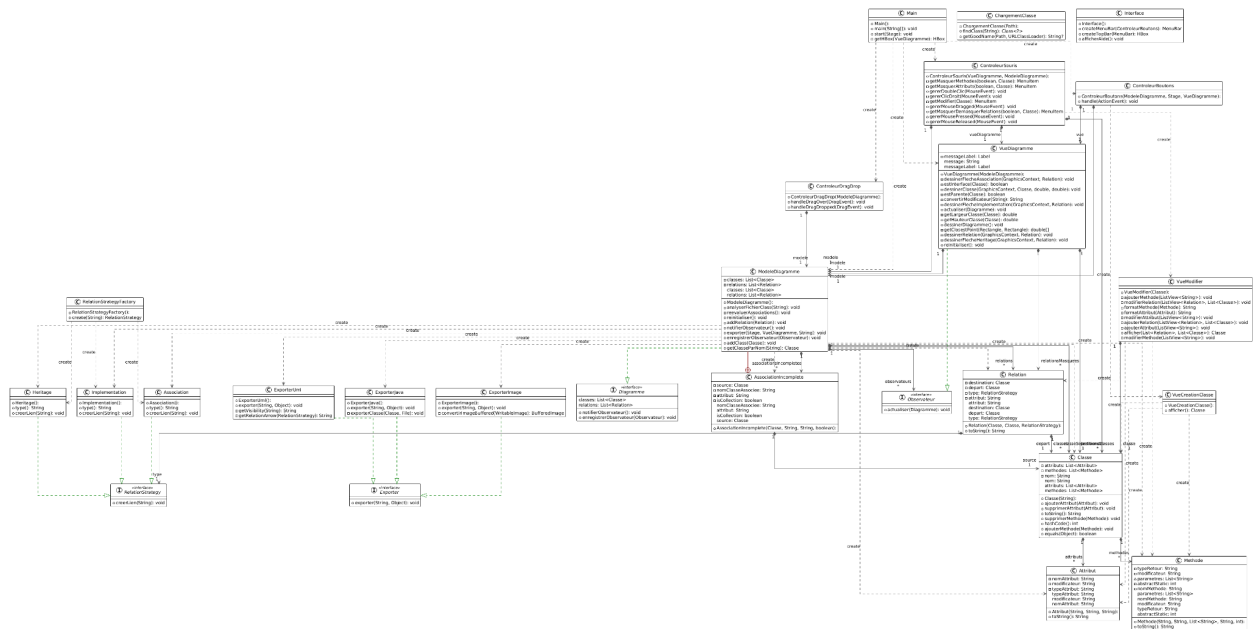
SOMMAIRE:

Liste des fonctionnalités:.....	1
Diagramme de classe final:.....	2
Répartition du travail entre étudiants:.....	2
Élément original du projet:.....	4
Éléments modifiés par rapport à l'étude préalable:.....	4
Patrons de conceptions / architecture:.....	4
Graphe de scène:.....	5
Organisation des vues:.....	5

Liste des fonctionnalités:

- **Drag and Drop** de fichiers `.class` depuis l'explorateur de fichiers.
- **Analyse des classes**, affichage du diagramme et gestion des relations à l'aide de l'introspection.
- **Déplacement des classes** dans l'interface de l'application.
- Manipulation des diagrammes : afficher ou masquer un attribut, une méthode ou une relation, suppression d'une classe.
- **Exportation** : exportation en PNG, en code UML ou en classes Java.
- **Réinitialisation** de la zone de diagramme.
- **Création de nouvelles classes** depuis le menu de l'application.
- **Modification des classes existantes** : ajout/suppression/modification de méthodes, attributs, relations.
- **Menu d'aide** pour expliquer l'utilisation de l'application.
- Implémentation de la fonctionnalité du **clic droit**.
- Implémentation de la fonctionnalité du **double clic**.

Diagramme de classe final:



Répartition du travail entre étudiants:

Itération	Nathan	Joey	Luka	Pacôme	Robin
Itération 1	Mise en place de la scène Javafx	Mise en place des différents patrons	Mise en place des squelettes des classes	Mise en place des différents patrons	Mise en place des différents patrons Diagramme de classe itération 1
Itération 2	Introspection, Analyse de classe Dessin visuel des classes en UML	Introspection, Analyse de classe Compte rendu Itération 1	Introspection, Analyse de classe	Introspection, Analyse de classe	Introspection, Analyse de classe Dessin visuel des classes en UML Diagramme de classe itération 2

Itération 3	Importation des points classes Amélioration Drag and Drop	Gestion des exceptions Tests Unitaires Compte rendu	Création de répertoires	Correctif bug(importation, conversion)	Conversion d'image Diagramme de classe itération 3
Itération 4	Correction bugs (importation) Placement aléatoires des classes importées	Mouvement des flèches pour éviter le croisement Génération code source Java	Gestion clic droit Afficher et masquer des éléments Suppression diagrammes	Affichage des flèches	Exportation PlantUML Diagramme de classe itération 4 Mouvement des classes avec un nouveau drag and drop
Itération 5	Debug Introspection finalisation Couleurs des classes Bugs flèches et relations	Debug Trello et tous les rapports Finalisation génération code source java	Debug Gestion clic droit Finalisation masquer les éléments et clic droits ajout et suppression d'attributs	Debug Bugs flèches et relations	Debug Diagramme de classe itération 5 Finalisation export plantuml Trello
Itération 6	Finalisation modification des classes Génération complète et abstraite des classes debug	Compte rendu Debug Dernier réglage code source Java(Bug avec les modificateurs) Style Application	Graphe de scène Compte rendu Style Application debug	Compte rendu Style Application debug	Diagramme de classe itération 6 Diagramme de séquence itération 6 Code source plantuml Finalisation modification des classes

Élément original du projet:

Nous avons intégré un système d'ajout et de modification de classes, une fonctionnalité complexe mais cruciale, qui rend l'application entièrement personnalisable.

Éléments modifiés par rapport à l'étude préalable:

Génération d'un diagramme de classe	Manipulation basique du diagramme	Exportation des diagrammes	Modifications avancées du diagramme	Génération des squelettes de classe	Finalisation et interface avancée
-------------------------------------	-----------------------------------	----------------------------	-------------------------------------	-------------------------------------	-----------------------------------

- Toutes les fonctionnalités mentionnées ont été réalisées.
- L'exportation génère désormais également le squelette en code Java.
- Nous avons implémenté la modification des relations, qui n'était pas prévue initialement, ainsi que la création de diagrammes à partir de zéro avec une interface dédiée.
- La finalisation de l'interface permet maintenant l'ajout du double clic pour masquer des éléments, et l'utilisation de couleurs pour différencier les interfaces.
- Plusieurs vues ont été créées : `VueDiagramme`, `VueCreationClasse`, et `VueModifier`.

Patrons de conceptions / architecture:

Patron MVC :

- **Modèle** : `ModeleDiagramme`
- **Contrôleurs** : `ControleurDragDrop` pour la gestion du drag & drop et `ControleurBoutons` pour les boutons du menu.
- **Vues** : `VueDiagramme` pour la visualisation du diagramme, `VueCreationClasse` pour la création de nouvelles classes, `VueModifier` pour la modification des classes.

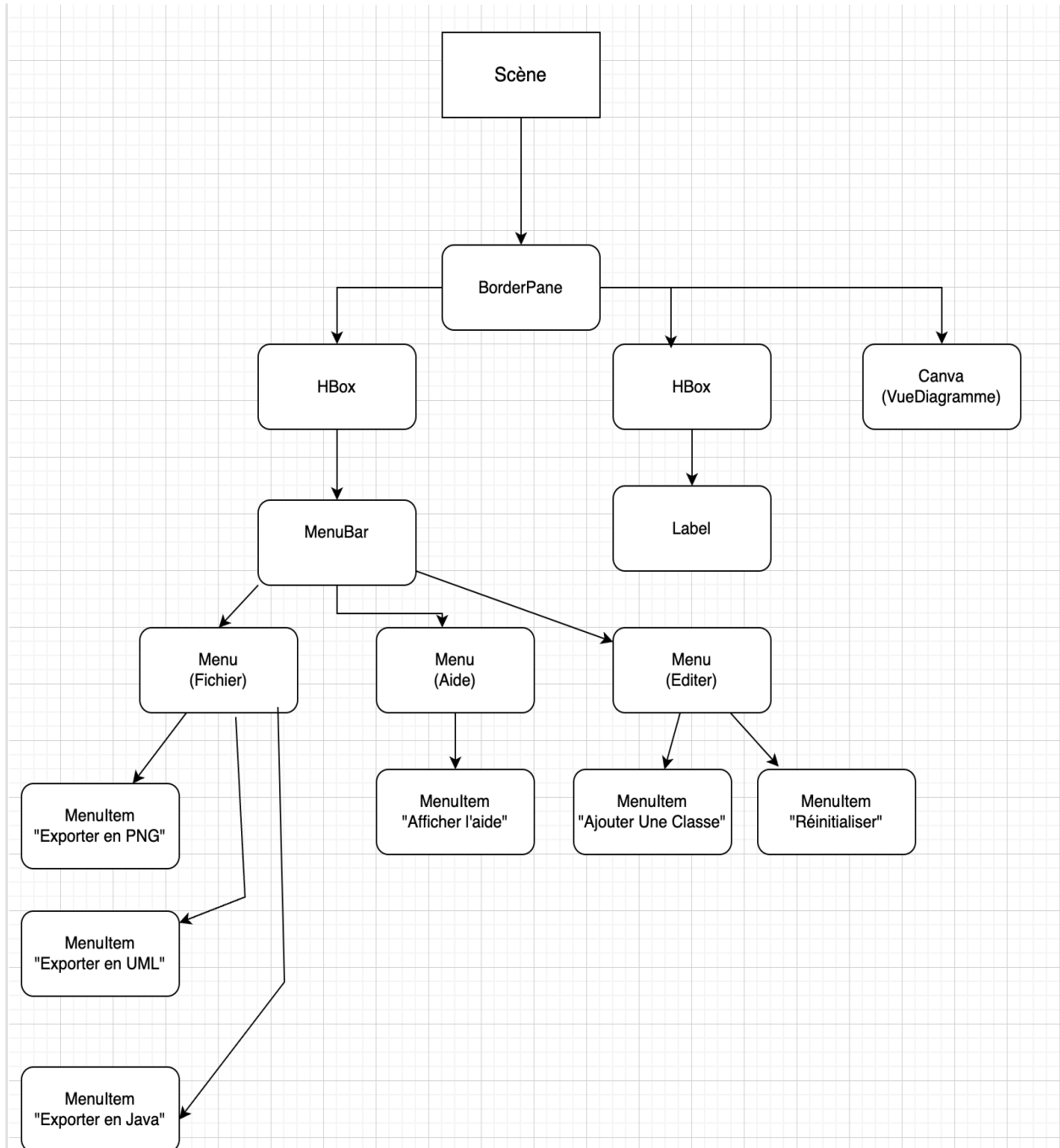
Patron Stratégie :

- La classe `Relation` possède un attribut `type` de type `RelationStrategy`, qui détermine le comportement de la relation en fonction de son type hérité (Héritage, Implémentation, Association).

Patron Fabrique :

- Utilisation d'une fabrique de relations via la classe `RelationStrategyFactory`.

Graphe de scène:



Organisation des vues:

Les vues suivent une architecture Modèle-Vue-Contrôleur.

- **VueDiagramme** : La vue principale pour afficher les diagrammes UML sur un **Canvas**.
- **VueCreationClasse** : Menu pour la création manuelle de classes.
- **VueModifieur** : Interface de modification des éléments existants (classes, attributs, méthodes, relations).

L'interface est mise à jour grâce à l'interface **Observateur**, qui réagit aux modifications dans le modèle. Le changement de vue est géré par les contrôleurs. Par exemple, lors de l'importation de classes ou du déplacement de celles-ci, le contrôleur met à jour le modèle, qui notifie la vue **VueDiagramme** pour redessiner le diagramme. Lors d'un changement de contexte (création ou modification d'une classe), un nouveau contrôleur est chargé pour afficher l'interface adéquate